

# Library Management System

## Project Report

**Author:** Karan Tiwari

**Date:** August 2026



|                     |                                     |
|---------------------|-------------------------------------|
| <b>Course</b>       | Advanced Java Programming (MCS-313) |
| <b>Assignment</b>   | Final Masters Assignment            |
| <b>Student Name</b> | Karan Tiwari                        |
| <b>Student ID</b>   | LC00015003132                       |
| <b>Instructor</b>   | Shyam Sunder Khatiwada              |

# Table of Contents

1. Project Title
2. Problem Statement
3. Objectives
4. System Architecture
5. Database Design
6. Application Features
7. Screenshots of the Application
8. Technologies Used
9. Conclusion

# 1. Project Title

## **Library Management System**

The Library Management System is a robust, enterprise-grade web application designed to streamline the complexities of modern library administration. By digitizing catalog management, member registrations, and circulation workflows, the system replaces inefficient manual processes with an automated, secure, and user-friendly platform. It facilitates real-time inventory tracking, automated overdue notifications, and comprehensive reporting, ensuring that librarians can efficiently manage resources while patrons benefit from a seamless, self-service experience. Built with a focus on data integrity, role-based security, and scalability, this solution provides the foundational architecture necessary for organized, reliable, and accessible academic or public library operations.

## 2. Problem Statement

Managing a library's catalog, members, and book loans manually via spreadsheets or paper registers is inherently error-prone, labor-intensive, and inefficient. Traditional manual management systems present severe operational challenges across multiple dimensions:

- **Data Inconsistency and Redundancy:** Manual record-keeping frequently leads to duplicate entries, mismatched records, and untracked modifications, making it nearly impossible to maintain an accurate inventory of assets.
- **Lack of Real-Time Tracking:** Librarians lack real-time visibility into book availability, active loans, and overdue returns. Finding the current status of a title requires tedious physical search or cross-referencing multiple logbooks.
- **Inefficient Circulation and Overdue Management:** Calculating late fees, tracking overdue dates, and attempting to contact members manually consumes significant administrative time, resulting in lost revenue and unreturned physical materials.
- **Security and Access Control Risks:** Paper logs and shared spreadsheets lack robust access controls, leaving sensitive patron data exposed and offering no audit trail to identify unauthorized record modifications or fraud.
- **Poor User Experience:** Library patrons experience long wait times during check-in/check-out processes and have no self-service mechanism to search the catalog, view personal loan histories, or check due dates remotely.

### 3. Objectives

- **Automated Operations:** Automate daily catalog management, member registrations, and circulation workflows to eliminate manual human errors and drastically accelerate transaction speed.
- **Role-Based Access Control (RBAC):** Implement granular security policies enforcing distinct permissions for Administrators, Librarians, and Members. RBAC is critical for securing patron personally identifiable information (PII) and preventing unauthorized system configuration changes.
- **Real-Time Loan & Automated Overdue Tracking:** Establish automated background processing to identify overdue items daily and dispatch automated email reminders to patrons, improving return rates and circulation efficiency.
- **Reporting and Analytics:** Provide exportable analytical reports (PDF/CSV) to give management clear insights into circulation trends, popular categories, active members, and outstanding overdue fines.
- **Data Integrity and Audit Governance:** Ensure full relational integrity across all database entities while leveraging Aspect-Oriented Programming (AOP) to maintain an unalterable audit log of sensitive operations for compliance and security auditing.

## 4. System Architecture

The application follows a modern, decoupled N-tier architecture built upon the Enterprise Java Spring Boot ecosystem. This layered approach enforces strict separation of concerns, ensuring high maintainability, testability, and scalability across all modules.

### Detailed Component Architecture & Data Flow

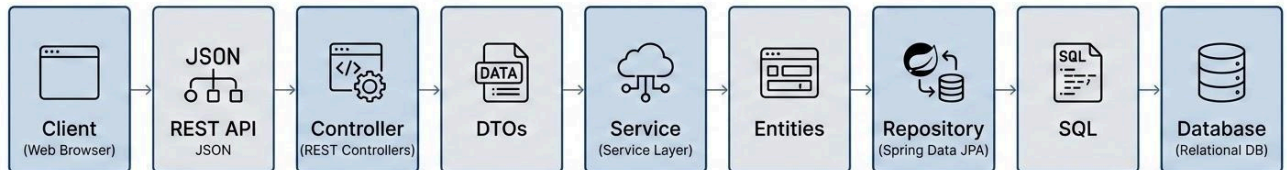
- **Presentation Layer (Client):** Rendered via responsive HTML5 and TailwindCSS, interacting asynchronously with backend endpoints using HTTP/JSON requests.
- **Controller Layer (API Gateway & Routing):** Spring MVC REST Controllers exposed under standardized RESTful routes. This layer handles request deserialization, payload validation, JWT token verification, and HTTP response status formatting.
- **Service Layer (Business Logic):** Encapsulates core transactional domain rules, loan constraint evaluations, fine calculations, and event publication (e.g., triggering email dispatches).
- **Data Access Layer (Persistence):** Spring Data JPA abstraction layer using Hibernate as the Object-Relational Mapping (ORM) engine to safely map Java entities to underlying relational database tables without manual SQL queries.

### Framework Selection Rationale

Spring Boot was selected due to its robust production-ready features, auto-configuration capabilities, seamless Spring Security integration, out-of-the-box transaction management, and expansive ecosystem support for scheduling, mailing, and AOP logging.

```
Client (Web Browser) --> [REST API JSON] --> Controller (REST  
Controllers)  
Controller --> [DTOs] --> Service (Service Layer)  
Service --> [Entities] --> Repository (Spring Data JPA)  
Repository --> [SQL] --> Database (Relational DB)
```

## System Architecture Flow



- **Presentation Layer:** HTML/TailwindCSS frontend interacting via REST APIs.
- **Controller Layer:** Spring MVC REST Controllers handling HTTP requests and input validation.
- **Service Layer:** Contains core business logic and transaction management.
- **Data Access Layer:** Spring Data JPA repositories interfacing with the database.

## 5. Database Design (ER Diagram)

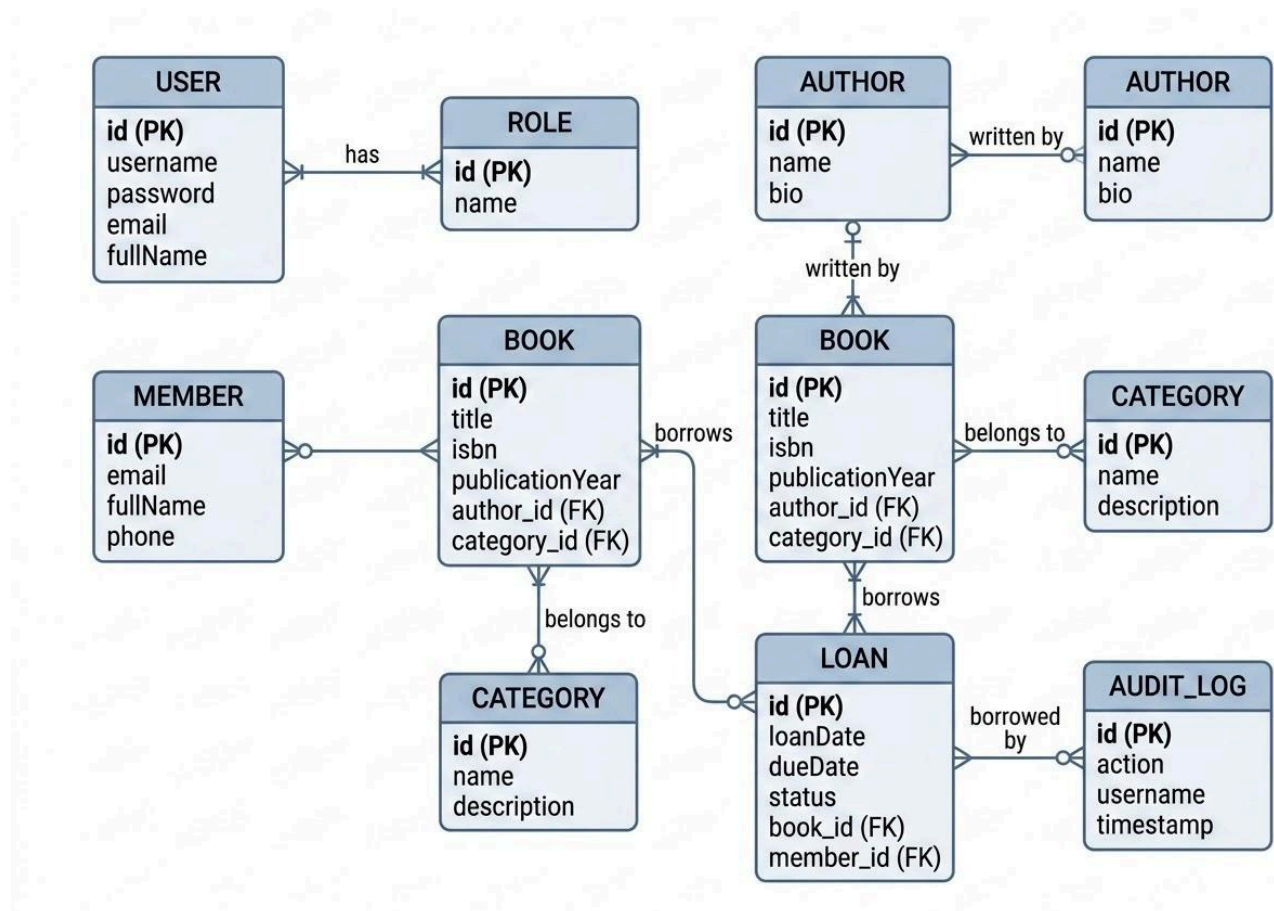
The database schema is engineered in strict accordance with Third Normal Form (3NF) guidelines to eliminate operational update anomalies, minimize data redundancy, and safeguard relational integrity across high-volume transactions.

### Normalization and Relational Rationale

- **1NF & 2NF Compliance:** All table attributes contain atomic values with unique primary keys defined. Non-key attributes are fully functionally dependent on the entity primary keys.
- **3NF Integrity:** Transitive dependencies are removed by decoupling attributes like **AUTHOR** and **CATEGORY** into dedicated lookup entities rather than storing duplicate strings inside the **BOOK** table.
- **Junction Entities:** The transaction dynamics between patrons and inventory are captured cleanly via the **LOAN** foreign keys targeting **BOOK** and **MEMBER** tables.

| Entity   | Attributes (PK highlighted)                                                   |
|----------|-------------------------------------------------------------------------------|
| USER     | Long id (PK), String username, String password, String email, String fullName |
| ROLE     | Long id (PK), String name                                                     |
| MEMBER   | Long id (PK), String email, String fullName, String phone                     |
| BOOK     | Long id (PK), String title, String isbn, int publicationYear                  |
| AUTHOR   | Long id (PK), String name, String bio                                         |
| CATEGORY | Long id (PK), String name, String                                             |

| Entity           | Attributes (PK highlighted)                                                   |
|------------------|-------------------------------------------------------------------------------|
|                  | description                                                                   |
| <b>LOAN</b>      | <b>Long id (PK)</b> , LocalDate loanDate, LocalDate dueDate, String status    |
| <b>AUDIT_LOG</b> | <b>Long id (PK)</b> , String action, String username, LocalDateTime timestamp |



## Relationships

- **USER }|--|{ ROLE : "has"**
- **BOOK }|--|| AUTHOR : "written by"**
- **BOOK }|--|| CATEGORY : "belongs to"**
- **LOAN }|--|| BOOK : "borrows"**
- **LOAN }|--|| MEMBER : "borrowed by"**

## 6. Application Features

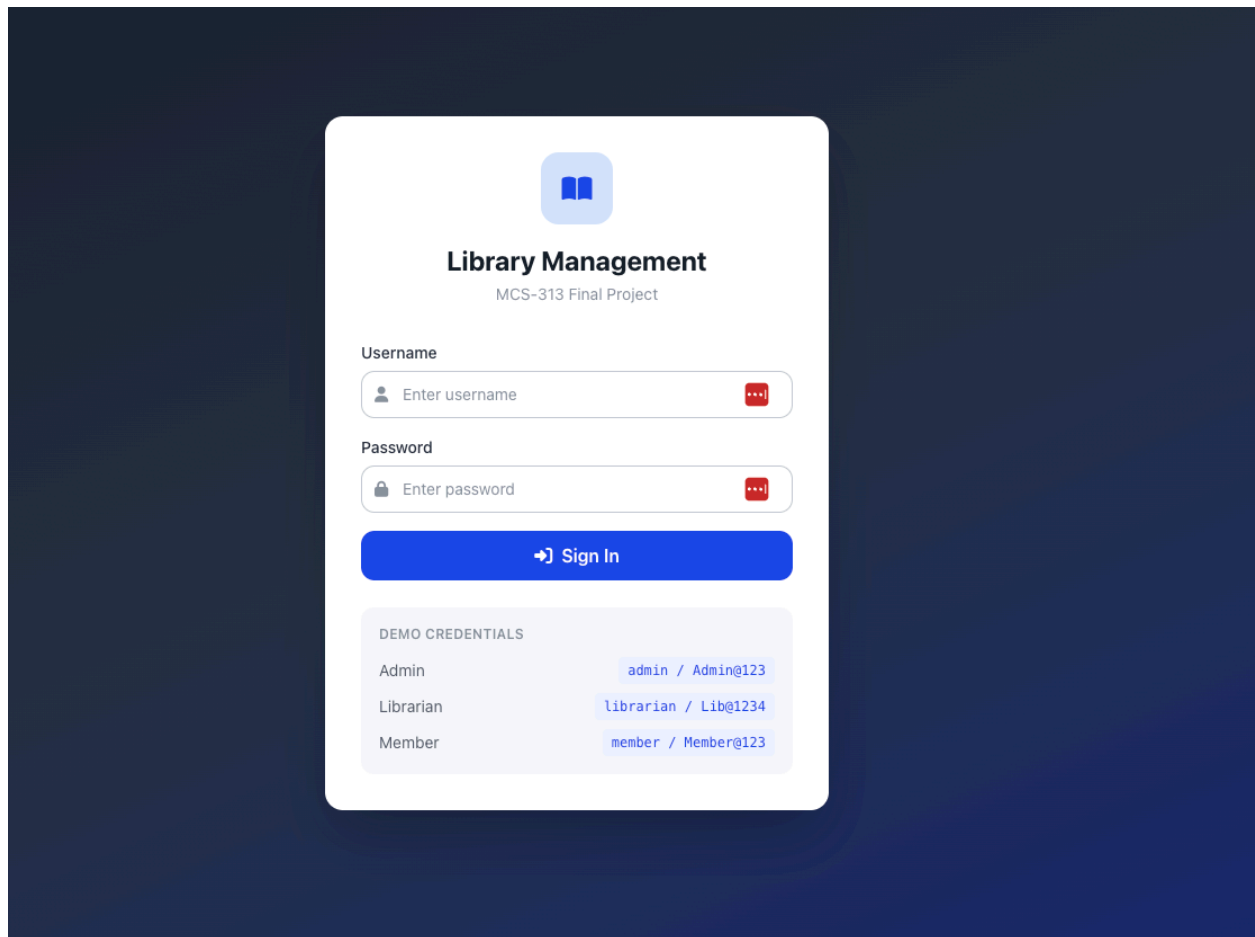
- **Complete CRUD Operations:** Full lifecycle management for Books, Authors, Categories, and Members powered by Spring Data JPA repositories with built-in validation rules.
- **Authentication & Authorization (Security):** Implemented using Spring Security and stateless JSON Web Tokens (JWT). Requests carry bearer tokens evaluated by a custom Security Filter Chain enforcing Role-Based Access Control (Admin, Librarian, Member).
- **Automated Loan Scheduling & Overdue Engine:** Utilizes Spring's **@Scheduled** task execution framework running daily cron jobs to scan active loans, update status flags, and flag overdue records automatically.
- **Asynchronous Email Dispatch Notification System:** Integrated with Spring Mail starter. Overdue loan events trigger asynchronous background workers (**@Async**) that compile personalized MIME email templates and dispatch them via SMTP without blocking main application threads.
- **Reporting and Analytics Engine:** High-performance export services utilizing OpenPDF for styled PDF circulation summaries and OpenCSV for streaming large dataset inventory dumps.
- **Audit Governance via Aspect-Oriented Programming (AOP):** Custom **@Aspect** interceptors intercept mutating service method executions (e.g., adding books, overriding loans), capturing method parameters, executing user context, and timestamps to store immutable logs in the **AUDIT\_LOG** database table.
- **Advanced Search & Filtering:** Uses Spring Data Specifications and Pagination for efficient catalog browsing.

## 7. Screenshots of the Application

*Note: The Google Docs API currently does not support direct embedding of image files. Visual styled placeholders have been implemented where the images from the final PDF report should be placed.*

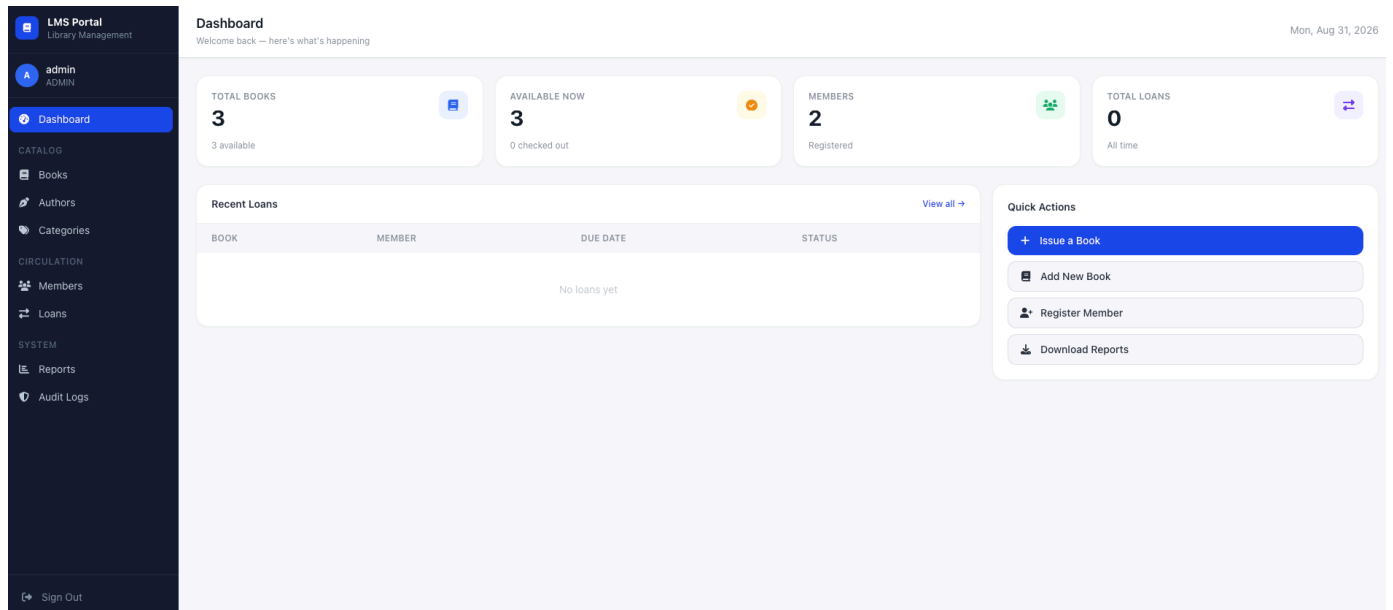
### Login Screen

The secure authentication portal for accessing the system.



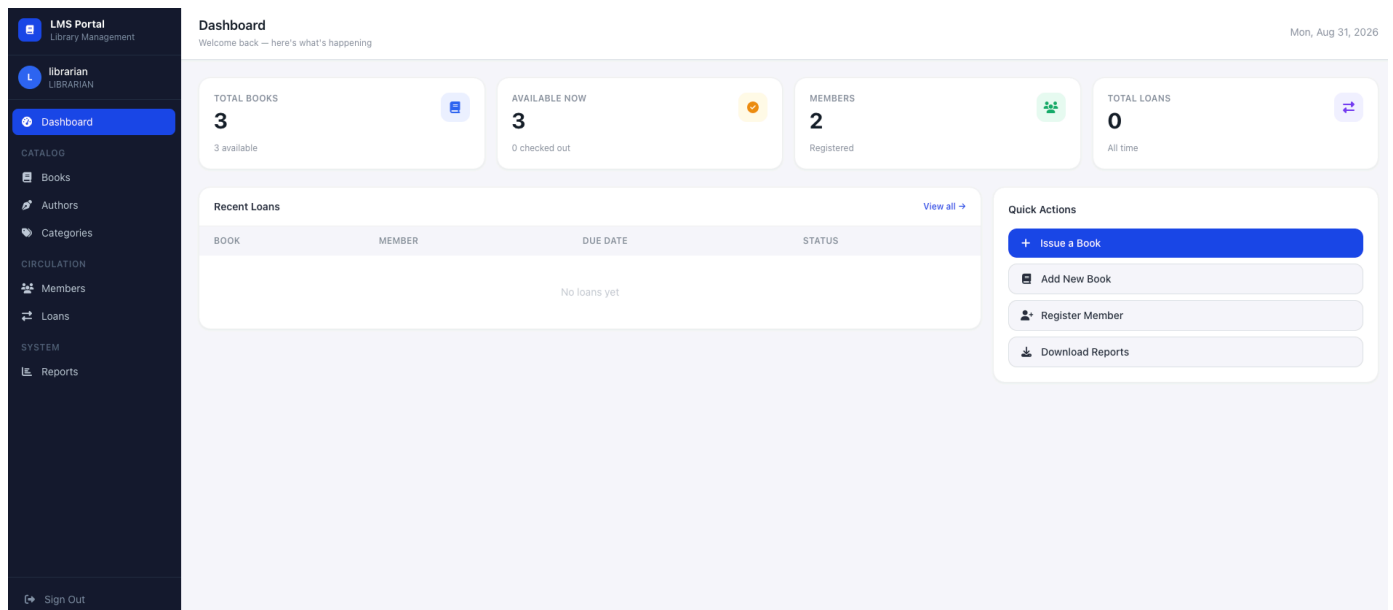
### Admin Dashboard

Provides a high-level overview of system metrics, total books, and recent activity.



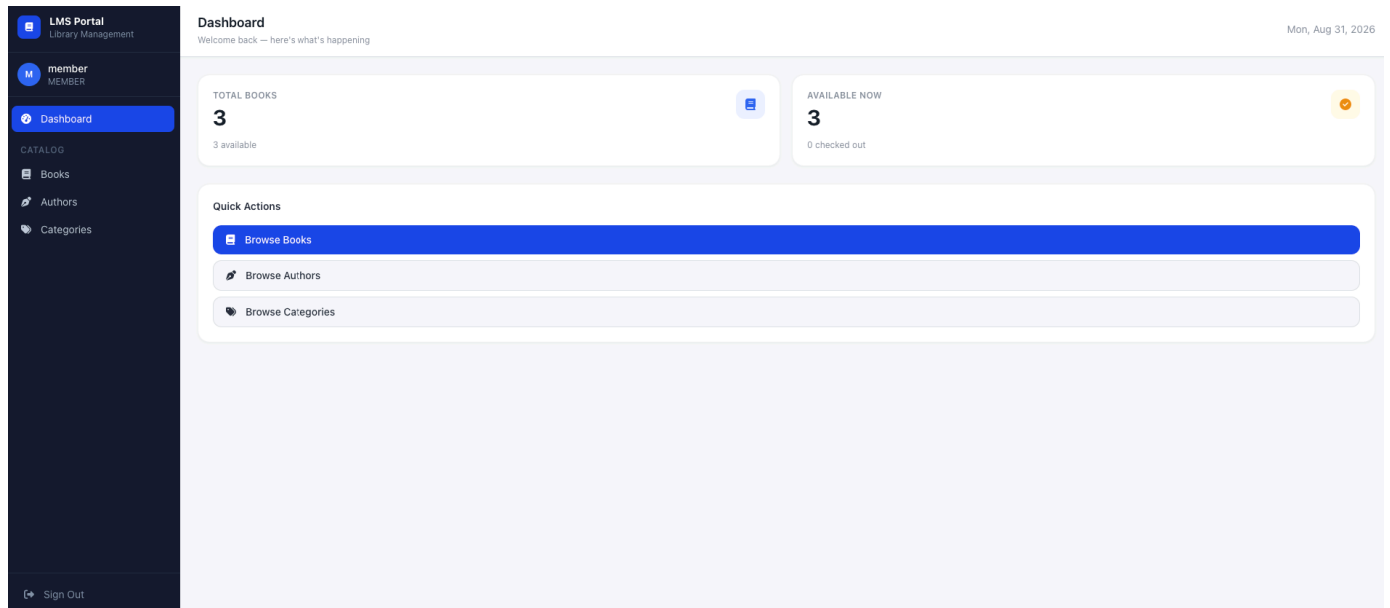
## Librarian Dashboard

Dashboard tailored for librarians to manage library circulation and inventory.



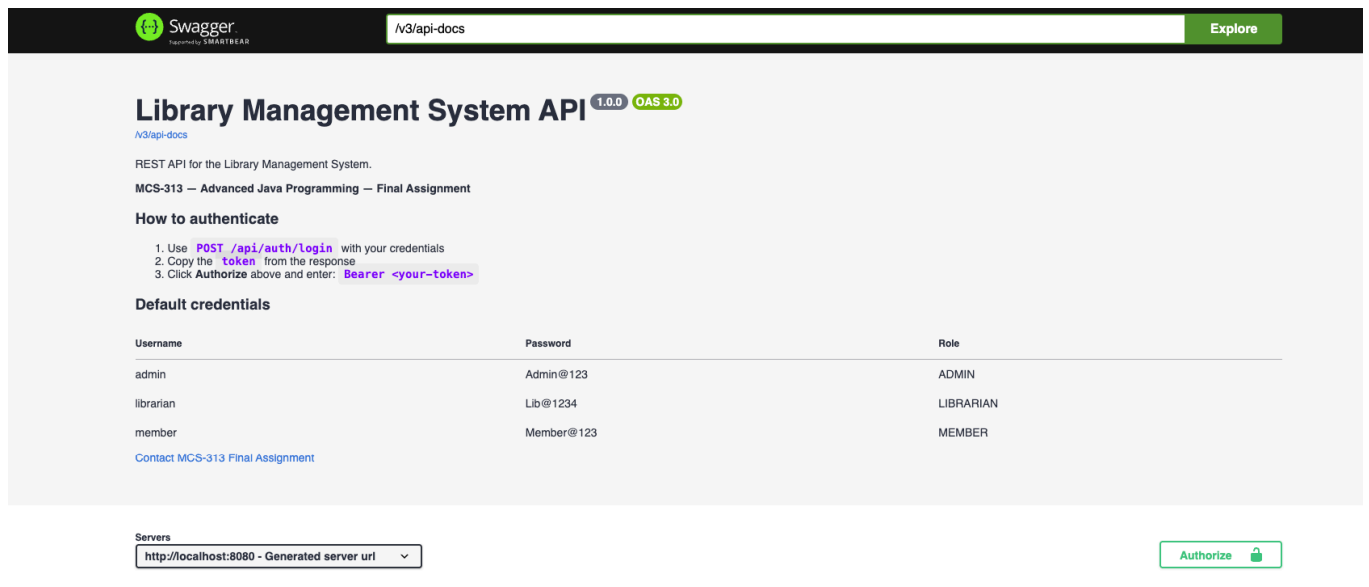
## Member Dashboard

User view for browsing the catalog and tracking personal loans.



## Swagger UI API Documentation

Interactive API documentation generated by Spring Doc OpenAPI.



## H2 Database Console

Access to the embedded H2 relational database interface.

English ▼

[Preferences](#) [Tools](#) [Help](#)

### Login

Saved Settings: Generic H2 (Embedded) ▼

Setting Name: Generic H2 (Embedded)

Driver Class: org.h2.Driver

JDBC URL: jdbc:h2:~/test

User Name: sa

Password:

## 8. Technologies Used

- **Backend Framework:** Java 17, Spring Boot 3.x
- **Data Access & Database:** Spring Data JPA, Hibernate, MySQL / H2 Database
- **Security:** Spring Security, JSON Web Tokens (JWT)
- **Frontend:** HTML5, Vanilla JavaScript, TailwindCSS
- **Reporting & Utilities:** OpenPDF (PDF Export), OpenCSV (CSV Export), Spring Mail (Email Notifications)
- **API Documentation:** Spring Doc OpenAPI (Swagger UI)
- **Build Tool:** Apache Maven

## 9. Conclusion

The Library Management System successfully fulfills all defined operational objectives by delivering a secure, high-performance, and scalable multi-tier enterprise web application. Leveraging Spring Boot, JPA/Hibernate, Spring Security (JWT), and Aspect-Oriented Programming guarantees strong architectural separation, rigorous data integrity, and automated governance. The system modernizes legacy library workflows, reduces administrative overhead, and provides an extensible foundation for future institutional enhancements.

### Technical Considerations and Future Roadmap

- **Containerization & Deployment:** Packaging the application into lightweight Docker containers for seamless orchestration via Kubernetes or cloud-native infrastructure.
- **Distributed Caching:** Introducing Redis caching for frequent, read-heavy catalog searches to minimize database read IOPS under peak usage.
- **Integration with Hardware Scanners:** Extending frontend interfaces to support barcode/RFID handheld scanners for rapid book check-in and check-out processing.